A Three-Stage SFT+GRPO Pipeline for Mathematical Reasoning on Qwen2.5-1.5B

Course Project Report

Group X*

April 19, 2026

Abstract

We present a three-stage training pipeline that takes the open-source Qwen2.5-1.5B base model [1] and turns it into a competent mathematical reasoner using only verifiable final answers as reward. The pipeline starts with a small, strictly filtered Supervised Fine-Tuning (SFT) set of 1 298 high-quality reasoning traces, then continues with large-scale Reinforcement Learning (RL) using either GRPO [2] or DAPO [3] on up to 8.1×10^4 verifiable problems drawn from the DeepMath-103K collection [4]. We design a multi-component reward that combines exact-match correctness, equivalence-aware matching, output-format shaping, and several penalties for repetition, role leakage, trailing content, and length overflow. Across five reasoning and general benchmarks, the final RL checkpoint lifts `GSM8K` [5] from 0.012 to 0.704, `MATH-500` [6, 7] from 0.014 to 0.476, `TheoremQA` [8] from 0.058 to 0.278, while `HellaSwag` [9] accuracy increases slightly ($0.502 \rightarrow 0.511$) and 5-shot `MMLU` [10] accuracy stays flat within standard error ($0.602 \rightarrow 0.600$). The results show that a verifiable-reward RL pipeline on a 1.5B base can produce large gains on mathematical reasoning while preserving general knowledge ability. All code, prompts, evaluation configs, and saved samples are released with the project.

Keywords: large language models, reinforcement learning, GRPO, DAPO, mathematical reasoning, verifiable rewards, evaluation.

1 Introduction

Motivation. Recent results on mathematical reasoning have shown that small open-source language models, originally trained only with next-token prediction, can be substantially improved by reinforcement learning with verifiable rewards [2, 3, 11]. The recipe is now widely agreed upon: (i) start from a competent base model, (ii) teach the model the required answer format with a small supervised fine-tuning step, and (iii) optimise a verifiable correctness signal on a much larger pool of problems whose final answers can be checked automatically.

This report documents a course-project implementation of that recipe on a single 1.5B-parameter base model, Qwen2.5-1.5B [1]. We deliberately keep the moving parts small: one base model, one SFT subset, one main RL data pool with two scales, two RL algorithms (GRPO and DAPO), and one shared evaluation suite. The goal is not to set a new state-of-the-art number, but to (a) reproduce the reported pattern of "small SFT followed by larger verifiable-reward RL" end-to-end, (b) study how each stage moves five evaluation benchmarks, and (c) check whether the mathematical training causes any regression on general knowledge tasks.

*Replace with author names, IDs, course code, and affiliation.

Contributions. The concrete contributions of this report are:

1. **End-to-end pipeline.** A reproducible three-stage pipeline (base → SFT → GRPO/DAPO) on Qwen2.5-1.5B, with strict separation between a clean SFT set requiring full reasoning traces and a large RL-ready set requiring only verifiable final answers (Section 2).
2. **Composite reward function.** A reward shape that combines exact-match and equivalence-aware correctness with several format, cleanliness, and length-control terms. The exact contributions are listed in Section 4.
3. **Five-benchmark evaluation matrix.** A unified evaluation that covers three mathematical benchmarks (`GSM8K`, `MATH-500`, `TheoremQA`) and two general benchmarks (`HellaSwag`, `MMLU`), all run through *lm-eval-harness* [12] with the same backend and decoding configuration. The complete numerical table is in Section 6.
4. **Failure-mode taxonomy.** A short qualitative analysis on saved `GSM8K` and `TheoremQA` samples that classifies the remaining errors into four recurring patterns (Section 8).

Outline. Section 2 describes data preparation and the four data layers. Section 3 explains the three-stage training design. Section 4 details the composite reward. Section 5 fixes the experimental setup and shows training-time dynamics for the two RL variants. Section 6 reports the main benchmark table. Section 7 focuses on whether mathematical training degrades general ability. Section 8 analyses the remaining failure modes, and Section 9 summarises and lists limitations.

2 Data

2.1 Source corpus

All training data originate from the DeepMath-103K collection [4], distributed in our project as `data/raw/deepmath_103k_raw.jsonl`. Each record contains a problem statement, a final reference answer, up to three candidate "R1-style" solution traces, a difficulty score, and a topic path. A minimal example is shown below.

```
{
  "question": "Evaluate the limit: ...",
  "final_answer": "0",
  "r1_solution_1": "Okay, so I have this limit to evaluate...",
  "r1_solution_2": "...",
  "r1_solution_3": "...",
  "difficulty": 4.5,
  "topic": "Mathematics -> ..."
}
```

Listing 1: A raw record from `deepmath_103k_raw.jsonl`.

Two distinct downstream artefacts are extracted from this source: a small SFT set that requires a clean reasoning trace, and a much larger RL-ready set that requires only a verifiable final answer.

2.2 SFT data: `strict_main`

The SFT subset is created by keeping only records whose problem, final answer, and at least one candidate trace all pass strict cleanliness filters. The resulting set, `strict_main`, contains **1 298** training samples and **169** validation samples. Each sample is then exported in a ShareGPT-style `messages` format, where the user message contains the math problem under a unified instruction template, and the assistant message contains a cleaned reasoning trace ending in a single

`Final Answer:` line. Trace cleaning includes (i) removing `</think>` markers, (ii) dropping repeated `Final Answer` headings or duplicated answer lines, (iii) stripping unnecessary `\boxed{}` wrappers, and (iv) appending a single normalised `Final Answer: <answer>` line.

2.3 RL-ready data

The RL data is generated by `data/build_rl_ready_dataset.py`, which relaxes the SFT requirement of "must have a complete reasoning trace" to "the question is well-formed and the final answer can be verified". Three variants are kept:

Name	Train	Valid	Notes
`rl_strict`	81 463	10 207	strict length limits, main RL pool
`rl_relaxed`	81 478	10 208	relaxed length limits, ablation pool
`bigmath_verified_grpo`	194 274	24 489	alternative larger pool, not used here

Table 1: The three RL-ready variants. This report uses `rl_strict` and a balanced sub-sample of it.

2.4 The 24K balanced sub-sample

To control training time on a single GPU we additionally derive a **24K balanced sub-sample** from `rl_strict`, called `sampled24k`. The sampling has two stages:

1. *Difficulty buckets.* Examples are split into four buckets by the `difficulty` field: `low` ≤ 3.0, `mid` ≤ 6.0, `high` ≤ 8.0, and `very_high` > 8.0 (or missing). Each bucket gets a fixed quota: 2 400 / 10 800 / 7 200 / 3 600.
2. *Task-type stratification within bucket.* Inside each bucket, the per-task-type quota follows the original task-type proportion of that bucket, with leftover quota redistributed proportionally if a task type is short of samples.

The validation split is the entire `rl_strict_valid` (10 207 examples). A fixed random seed (`seed=42`) makes the sample reproducible. The resulting 24K mixture is dominated by `algebra` and `calculus`, with non-trivial coverage of `geometry`, `number_theory`, and `probability`.

2.5 Data layers

For clarity it is useful to separate the data into four layers, summarised in Table 2.

Layer	Purpose	Example file
1. Raw download	Original DeepMath records	`data/raw/deepmath_103k_raw.jsonl`
2. Cleaned middle	Normalised question, answer, candidates	`data/processed/filtered_strict_train.jsonl`
3. SFT export	ShareGPT `messages` format	`data/processed/sft_strict_main_train.json`
4. RL export	Single prompt + verifiable target answer	`data/processed/grpo_strict_main_train.jsonl`

Table 2: Four data layers used in this project.

Remark 1. The clean separation between layer 3 and layer 4 is what makes the SFT and RL stages compatible. Both stages use the same instruction template and the same final-answer extraction logic, so a model trained at layer 3 can be directly continued with RL at layer 4.

3 Method

3.1 Three-stage training design

Our pipeline has three sequential stages, summarised in Table 3.

Stage	Purpose	Train size	Data
S1: base reference	Provide a starting score with no training	0	—
S2: SFT	Teach answer format, reasoning style, output schema	1 298	`strict_main`
S3: GRPO/DAPO	Optimise verifiable correctness on a large pool	up to 81 463	`rl_strict` or `sampled24k`

Table 3: Three training stages used in this report. S1 is evaluation only.

3.2 Base model

We use QWEN2.5-1.5B [1] as the base. Earlier internal tests on a smaller QWEN2.5-0.5B variant proved unstable across both SFT and RL, so all numbers in this report use the 1.5B variant. We keep three model artefact types throughout the project, listed in Table 4.

Form	Identifier	Use
`base`	QWEN2.5-1.5B	Reference for S1
`sft`	`outputs/sft/strict_main_qwen25_15b_base_bs32_ga1`	LoRA SFT result, S2 output
`sft merged`	`qwen base sft merged`	Full-parameter base for GRPO/verl

Table 4: Three model forms used during training and evaluation.

3.3 Stage 2: SFT

The S2 stage performs supervised fine-tuning on the `strict_main` ShareGPT export. We use a LoRA adapter with rank 16, batch size 32, and one gradient-accumulation step on a single GPU. The training objective is the standard next-token cross-entropy on the assistant turn only. The aim is purely to teach the model:

- the math instruction template (problem + requirements);
- the structure of a concise step-by-step reasoning trace;
- the unique `Final Answer: <answer>` closing line.

After S2, the LoRA adapter can either be evaluated directly (rows "qwen base sft" below) or merged into the base weights to provide a starting point for full-parameter RL.

3.4 Stage 3: GRPO and DAPO

Stage 3 applies a verifiable-reward RL algorithm. We use two related algorithms:

- **GRPO** [2], which estimates the advantage by standardising rewards within a group of G rollouts that share the same prompt, removing the need for a separate value network.
- **DAPO** [3], which extends GRPO with decoupled clipping bounds and a dynamic-sampling strategy that drops trivial or impossible prompts on the fly. We use the open-source *verl* framework [13] to run both algorithms on a single GPU.

For a prompt q with reference answer a^* and a group of G sampled completions $\{o_i\}_{i=1}^{G}$, GRPO uses the standardised advantage

$$\hat{A}_i \;=\; \frac{R(q, o_i, a^*) - \mu_R}{\sigma_R + \varepsilon}, \qquad i = 1, \ldots, G, \tag{1}$$

where μ_R and σ_R are the mean and standard deviation of the group rewards, ε is a small constant for numerical stability, and $R(\cdot)$ is the composite reward defined in Section 4.

The policy is then updated with a PPO-style clipped objective

$$\mathcal{L}_{\text{policy}}(\theta) \;=\; -\mathbb{E}\Big[\min\Big(\rho_i\,\hat{A}_i,\; \text{clip}(\rho_i, 1-\epsilon, 1+\epsilon)\,\hat{A}_i\Big)\Big], \quad \rho_i \;=\; \frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)}. \tag{2}$$

A KL term against the SFT reference policy is added with a small coefficient. The DAPO variant replaces the symmetric clip ϵ with the asymmetric pair $(\epsilon_{\text{low}}, \epsilon_{\text{high}})$ and filters prompts whose group reward is degenerate.

3.5 Checkpoints reported

Within stage 3 we evaluate three named checkpoints: `grpo sampled24k`, `dapo`, and `grpo`. The first uses the 24K balanced sub-sample, the latter two use the full `rl_strict` pool of 81 463 problems. Both `dapo` and `grpo` correspond to mid-training checkpoints (step 150) rather than the very last training step, since their accuracy curves had already plateaued.

4 Reward Function

4.1 Design philosophy

Early experiments using a single $\{0, 1\}$ correctness reward produced unstable training: gradient signals were sparse and the policy quickly learnt to exploit boundary cases (truncated answers, role-token leakage, or extremely long output that hides the answer). Following the lesson patterns in DeepSeek-Math [2] and DAPO [3], we use a *composite* reward that mixes positive shaping terms for correct and clean outputs with several explicit penalties.

The total reward $R(q, o, a^*)$ for a prompt q, completion o, and reference answer a^* is the sum of all components defined in Table 5.

4.2 Components

Component	Score	When applied
r_{exact}	$+1.00$	extracted final answer is character-equal to a^* after normalisation
r_{equiv}	$+0.80$	extracted answer is mathematically equivalent to a^* but textually different
r_{fmt}	$+0.05$	output ends with a single well-formed `Final Answer:` line
r_{clean}	$+0.03$	no spurious trailing content after the answer line
`<answer>` bonus	0.00	reserved hook, currently disabled
p_{noans}	-0.12	no answer can be extracted from the output
p_{rep}	-0.08	visible repetition (n-gram or paragraph) detected
p_{role}	-0.08	role tokens (`user`, `assistant`, ...) leak into the output
p_{tail}	up to -0.20	long irrelevant text after the answer line
p_{len}	up to -0.05	output exceeds the length budget

Table 5: All reward components used in training. All shape terms are additive.

4.3 Formal expression

Let $\hat{a}(o)$ denote the answer extracted from the completion o. With the indicator $\mathbb{1}[\cdot]$ we can write

$$\begin{aligned} R(q, o, a^*) = {} & r_{\text{exact}} \cdot \mathbb{1}[\hat{a}(o) \equiv_{\text{str}} a^*] + r_{\text{equiv}} \cdot \mathbb{1}[\hat{a}(o) \equiv_{\text{math}} a^*] \\ & + r_{\text{fmt}} \cdot \mathbb{1}[\text{format ok}] + r_{\text{clean}} \cdot \mathbb{1}[\text{no tail}] \\ & - p_{\text{noans}} \cdot \mathbb{1}[\hat{a}(o) = \varnothing] - p_{\text{rep}} \cdot \mathbb{1}[\text{repetition}] \\ & - p_{\text{role}} - p_{\text{tail}}(o) - p_{\text{len}}(o), \end{aligned} \tag{3}$$

where the two correctness indicators are mutually exclusive (an exact match takes precedence over equivalence). The length penalty $p_{\text{len}}(o)$ starts at zero up to a budget of **256 tokens** and grows linearly beyond that, capped at -0.05.

4.4 Answer extraction and normalisation

Three answer formats are accepted in parallel: `Final Answer:` `<answer>`, `<answer>...</answer>`, and `\boxed{...}`. After extraction the answer string goes through a normalisation pass that (i) removes thousand separators in numbers, (ii) unifies $\sqrt{\cdot}$ written as `\sqrt{}` or as the Unicode glyph $\surd$, (iii) normalises common minus and division signs, (iv) accepts simple algebraic equivalents (e.g., 0.5 vs. 1/2), and (v) trims trailing text past the answer anchor when present. Equivalence matching is checked after normalisation only when the exact match fails.

4.5 Empirical observations

The composite design has two practical effects. First, removing p_{tail} in an ablation produced visible reward hacking: long noise after a correct answer line. Second, removing p_{len} slowly inflated average response length without a measurable accuracy gain, wasting decoding budget. Both penalties stayed in the final recipe.

5 Experimental Setup

5.1 Hardware and software stack

All training and evaluation runs were carried out on a single NVIDIA B200 GPU. The training stack uses `transformers`, `peft` (LoRA), and the *verl* framework [13] for GRPO and DAPO. SGLang is used as the rollout backend during RL. Evaluation is performed by *lm-eval-harness* [12] version `0.4.11`, with the HuggingFace backend for repeatability.

5.2 Training hyperparameters

Table 6 lists the main hyperparameters used in S2 and S3. The listed S3 values describe the RL configuration used in the main runs that produced the `dapo` and `grpo` checkpoints in the result table.

5.3 Evaluation protocol

Five benchmarks are evaluated under one shared protocol:

- **Mathematical:** `GSM8K` [5] (test split, `mathrl` task config with normalised answer extraction), `MATH-500` [6, 7] (the held-out 500-problem subset of MATH), and `TheoremQA` [8].

Hyperparameter	Value	Stage
LoRA rank / alpha	16 / 32	S2
Learning rate (AdamW)	2×10^{-4}	S2
Batch size / grad accum	32 / 1	S2
Train epochs	3	S2
Train batch size	512	S3
PPO mini-batch size	128	S3
Group size G (rollouts)	8	S3
Sampling temperature	1.0	S3
Max prompt length	1024 tokens	S3
Max response length	1024 tokens	S3
Length budget for p_{len}	256 tokens	S3
KL coefficient	1×10^{-3}	S3
PPO clip ϵ	0.20 (GRPO)	S3
DAPO clips $(\epsilon_{\text{lo}}, \epsilon_{\text{hi}})$	$(0.20, 0.28)$	S3

Table 6: Main training hyperparameters.

- **General:** `HellaSwag` [9] 0-shot multiple-choice accuracy, and `MMLU` [10] 5-shot multiple-choice accuracy aggregated across all 57 subjects (14 042 questions in total).

For mathematical benchmarks we apply the same chat template used during training and decode greedily with `temperature`=0, `max_gen_toks`=1024 (or 2048 for `TheoremQA`). For the general benchmarks the lm-eval-harness default loglikelihood scoring is used; the chat template is intentionally *not* applied to `MMLU` so that the answer-letter loglikelihood comparison is well defined.

5.4 Training-time dynamics

Figures 1 and 2 together suggest that GRPO and DAPO behave qualitatively similarly under our reward shape and hyperparameter range: both contract their output, then learn to produce an answer-line ending that consistently triggers r_{exact} or r_{equiv}, and both plateau in the same reward band. The choice between them is therefore more about engineering preferences than absolute performance, which is reflected in the close numbers reported in Section 6.

6 Main Results

6.1 Headline numbers

Table 7 reports the headline accuracy of all five model configurations on all five benchmarks. Figure 3 visualises the same numbers.

6.2 Stage-wise reading

The progression in Table 7 can be read in three blocks.

Base → SFT. The base model essentially fails on the math benchmarks under our instruction template (`GSM8K` = 0.012, `MATH-500` = 0.014, `TheoremQA` = 0.058), confirming that the failure is one of *format* rather than *capability*: the base model is not producing answers in the `Final Answer:` format that the `mathrl` extractor recognises. SFT on a small set of 1 298 cleaned traces is enough to lift all three math benchmarks by roughly 10× on `GSM8K` and `MATH-500` and roughly 3× on `TheoremQA`.

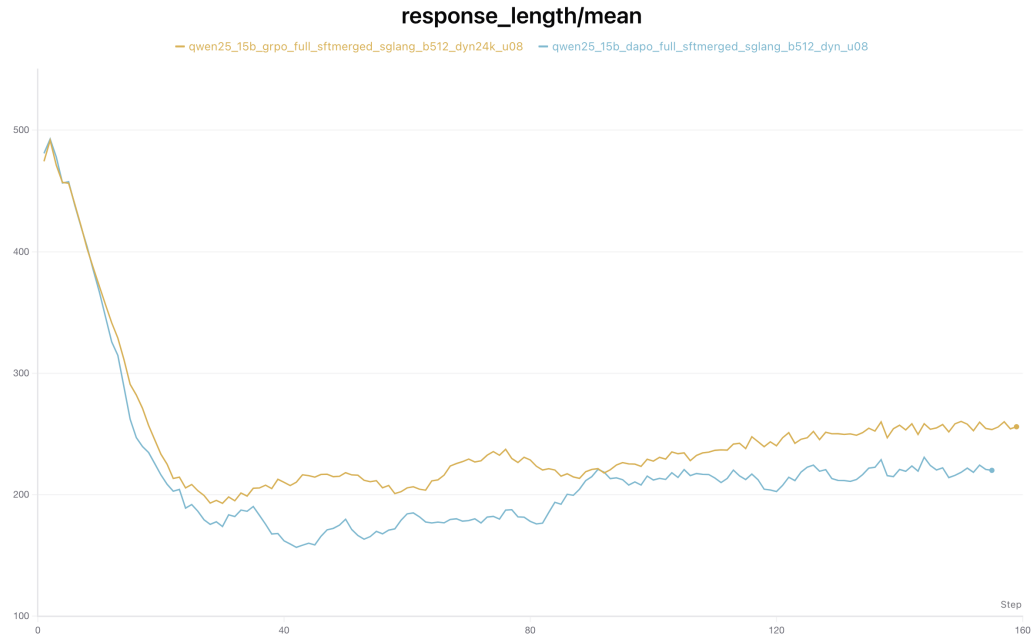

Figure 1: Mean response length during S3 training for GRPO (orange) vs. DAPO (blue). Both curves first contract from a long-output prior to a tighter regime in the first ∼20 steps, then re-expand modestly. GRPO settles around 250+ tokens, DAPO around 220+, indicating that GRPO preserves slightly longer reasoning traces under the same length budget.

Model stage	Train size	GSM8K	MATH-500	TheoremQA	HellaSwag	MMLU
qwen base	0	0.0121	0.0140	0.0576	0.5018	0.6019
qwen base sft	1 298	0.1638	0.1440	0.1673	0.5068	0.5993
grpo sampled24k	24 000	0.6884	**0.4760**	0.2396	0.5101	**0.6030**
dapo	81 463	0.7028	0.4180	0.2691	0.5105	0.6014
grpo	81 463	**0.7043**	0.4380	**0.2784**	**0.5112**	0.5998

Table 7: Main result table. Column *Train size* is the number of training examples actually consumed at the corresponding stage. Best number per benchmark in **bold**. The MMLU column is 5-shot accuracy across 57 subjects; standard error of the MMLU column is approximately ±0.0039, so all five MMLU numbers are statistically tied.

SFT → RL. The verifiable-reward RL stage produces a much larger second jump: `GSM8K` climbs from 0.164 to roughly 0.70, `MATH-500` from 0.144 to roughly 0.45, and `TheoremQA` from 0.167 to roughly 0.27. The two RL algorithms (GRPO, DAPO) are nearly tied on `MATH-500` but GRPO is consistently a bit ahead on `GSM8K` and `TheoremQA`. The 24K-sampled run sits between SFT and full-pool RL on `GSM8K` and `TheoremQA`, but is the strongest on `MATH-500`, suggesting that the difficulty-balanced 24K mixture is particularly well matched to the harder MATH-500 distribution.

Reward design effects. The strong `GSM8K` numbers are partly attributable to the *format* and *cleanliness* terms in Table 5: on saved samples, `Final Answer:` coverage stays at 1.0 for both `dapo` and `grpo`, with no role-token leakage observed. This matches the qualitative pattern in Section 8: format errors disappear after RL and most remaining failures are semantic.

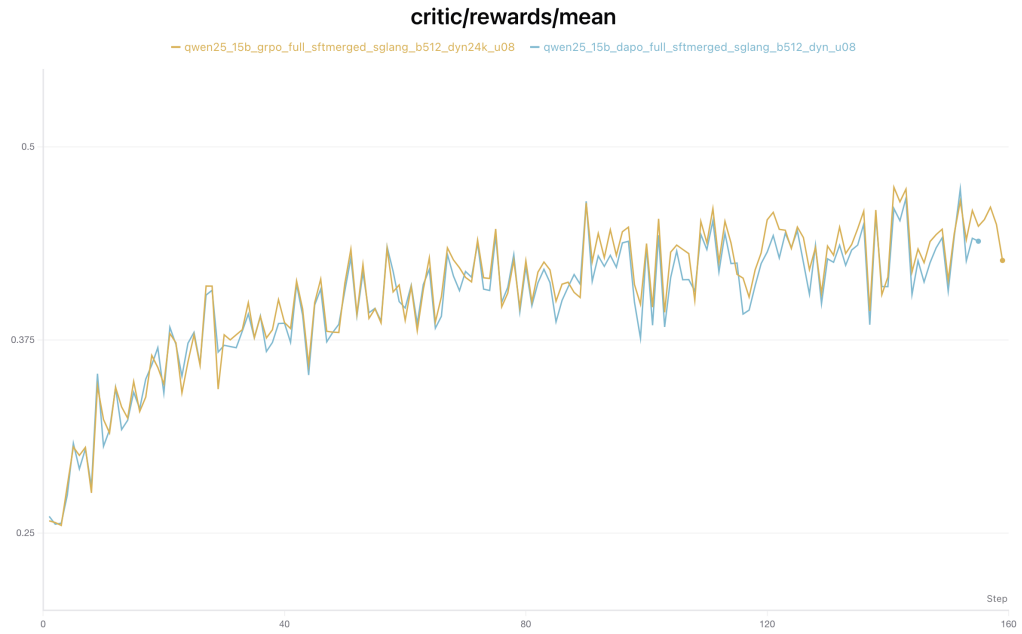

Figure 2: Mean reward μ_R during S3 training. Both algorithms climb from ≈ 0.25 to the 0.42–0.46 band and stay there. The two trajectories are very similar, with GRPO ending marginally higher.

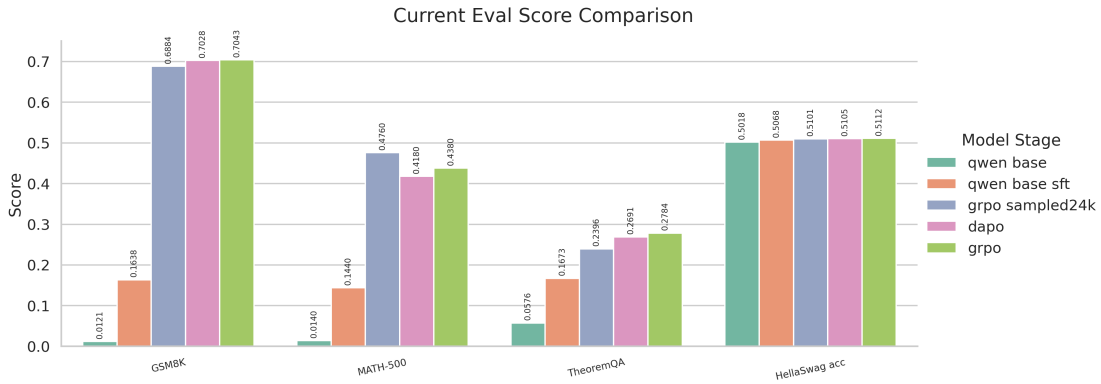

Figure 3: Per-benchmark accuracy across the five model stages.

6.3 Best-per-metric summary

- `GSM8K`: **grpo** (0.7043).
- `MATH-500`: **grpo sampled24k** (0.4760).
- `TheoremQA`: **grpo** (0.2784).
- `HellaSwag`: **grpo** (0.5112).
- `MMLU`: **grpo sampled24k** (0.6030, statistical tie).

7 Does Math RL Hurt General Ability?

A common concern with task-specific RL is that the policy gains on the target distribution at the cost of forgetting general capabilities. We check this on two general-purpose multiple-choice benchmarks: `HellaSwag` [9] and `MMLU` [10].

7.1 Numbers

Table 8 extracts the two general columns from Table 7 and adds the standard error from *lm-eval-harness* for both benchmarks.

Model stage	HellaSwag acc	MMLU acc (5-shot)
qwen base	0.5018	0.6019 ± 0.0039
qwen base sft	0.5068	0.5993 ± 0.0039
grpo sampled24k	0.5101	0.6030 ± 0.0039
dapo	0.5105	0.6014 ± 0.0039
grpo	0.5112	0.5998 ± 0.0039

Table 8: General-ability evaluation. `HellaSwag` is 0-shot; `MMLU` is 5-shot, aggregated across 57 subjects with 14 042 questions, weighted by subject size. Standard error is reported by *lm-eval-harness*.

7.2 Reading the table

HellaSwag. The HellaSwag accuracy moves *slightly upward* along the training pipeline, from 0.502 at the base to 0.511 at the final `grpo` checkpoint. The differences are small (less than 1.0 absolute percentage point) but the direction is monotonic, suggesting that the math pipeline at least does not damage commonsense multiple-choice judgement.

MMLU. The MMLU column is the more demanding general test. All five model stages land in the narrow band $[0.5993, 0.6030]$. With a per-row standard error of ± 0.0039, all five MMLU numbers are statistically indistinguishable. The pipeline consequently leaves multi-subject knowledge ability essentially unchanged — neither the small SFT step nor the much larger RL step measurably degrades it.

7.3 Why this matters

The combined picture from Table 7, Table 8, Fig. 1, and Fig. 2 is consistent with a clean interpretation: a verifiable-reward RL stage on a 1.5B base *can* deliver multi-fold accuracy gains on math reasoning while keeping general knowledge ability intact. A negative result on either general benchmark would have suggested that the reward shape is too narrow or that the RL update is over-fitting to surface-level math patterns. We do not see that here.

Remark 2. The MMLU evaluation is intentionally run *without* the math chat template: the lm-eval-harness MMLU task scores by comparing the loglikelihood of the four answer letters `A/B/C/D` under the model, and a chat template would change the conditioning text in a way that breaks comparability across the five models. The same applies to `HellaSwag`.

8 Failure Modes

The numerical gains in Section 6 leave a non-trivial gap to perfect accuracy. To understand what is left, we hand-inspected saved samples from `GSM8K` and `TheoremQA` produced by the `dapo` and `grpo` checkpoints. The qualitative analysis is the basis for the failure summary in Table 9; the full HTML listing is shipped at `analysis/failure_cases_dapo_grpo.html`.

8.1 Format vs. semantics

The most striking property of the late-stage checkpoints is that the *format* failures essentially disappear:

- `Final Answer:` coverage on the saved `GSM8K` batches is 1.0 for both `dapo` and `grpo`.
- No role-token leakage (e.g., stray `user:` or `assistant:` markers) was observed in the inspected outputs.
- Trailing-content violations are rare; when present, they are short polite sign-offs that the cleanliness penalty p_{tail} caught during training but did not eliminate at inference.

The remaining failures are therefore mostly *semantic*, which is what we want to characterise.

8.2 A four-way taxonomy

Table 9 groups the recurring error patterns observed in the inspected samples.

Failure type	**Where it appears**	**What it looks like**
semantic misread of local wording	`GSM8K`	The model maps a phrase to the wrong operation, e.g., reading an alternating discount as a geometric decay.
event-boundary state error	`GSM8K`	The model loses track of what gets reset after an interruption (a stop, a refill, a new day) and continues from the wrong state.
overconfident heuristic shortcut	`TheoremQA`	A familiar theorem or symmetry is applied without checking whether the preconditions actually hold for the current problem.
clean format, wrong content	`dapo`, `grpo`	The answer line is well-formed and the reasoning is fluent, but the underlying mathematical path is incorrect.

Table 9: Four recurring failure patterns observed on saved samples after RL training.

8.3 Examples

Example 1 (GSM8K, semantic misread)**.** A discount of *50% off, then 30% more off the discounted price* is sometimes read as a single 80% discount (i.e., additive) or as a geometric decay over many steps. The model writes a clean `Final Answer:` line, but the multiplicative chain 0.5×0.7 is dropped.

Example 2 (GSM8K, event-boundary error)**.** "She rests for one hour, then continues at the original pace" is treated as if the rest period also accumulates work, leading to a final quantity that is off by exactly the rest interval.

Example 3 (TheoremQA, heuristic shortcut)**.** A symmetry argument that holds for *spherically* symmetric inputs is applied to a *cylindrically* symmetric one, producing a final expression that is dimensionally correct but quantitatively wrong.

8.4 Implications for future work

Two practical implications follow:

- Adding a *precondition-check* reward bonus (or a chain-of-thought verifier in the reward function) could specifically attack the *overconfident heuristic shortcut* class on `TheoremQA`.
- Adding a *state-consistency* micro-reward (e.g., automatic re-evaluation of intermediate quantities) could attack the *event-boundary* class on `GSM8K`.

Both of these would require an external verifier beyond the current final-answer check, and are left as future work.

9 Conclusion

9.1 What we built

This report walked through a complete three-stage pipeline that takes Qwen2.5-1.5B from a base model that essentially fails on math benchmarks under our instruction template to a 1.5B-parameter model that solves $\sim 70\%$ of `GSM8K` and $\sim 45\%$ of `MATH-500`. The pipeline has three replaceable parts: (i) a small SFT step on **1 298** cleaned reasoning traces, (ii) a verifiable-reward RL step on up to **81 463** math problems using GRPO or DAPO, and (iii) a composite reward that mixes correctness, format shaping, and several length-and-cleanliness penalties.

9.2 Headline takeaways

1. *Most of the math gain comes from RL.* SFT alone explains only the first $\sim 10\times$ improvement on `GSM8K` and `MATH-500`. The much larger second jump comes from the verifiable-reward RL stage.
2. *GRPO and DAPO are nearly equivalent* under our reward shape and hyperparameter range. The numerical differences in Table 7 are smaller than the difference between the two different training pools.
3. *General ability is preserved.* Both `HellaSwag` and `MMLU` stay at the base level (`HellaSwag` drifts slightly upwards, `MMLU` stays statistically flat).
4. *Format failures disappear, semantic ones remain.* After RL, the inspected samples no longer show role-token leakage or truncated answer lines. The remaining errors are dominated by the four semantic patterns in Table 9.

9.3 Limitations

- *Scale.* A single 1.5B base model on a single GPU does not establish how the same recipe transfers to 7B+ regimes. Earlier internal experiments at 0.5B already showed that very small bases are fragile in both SFT and RL, suggesting non-linear interaction with scale.
- *Reward shape.* The reward function in Section 4 is hand-tuned. A more principled comparison against simpler rewards (e.g., pure $\{0, 1\}$ correctness or correctness + format only) would clarify which terms are necessary versus convenient.
- *Checkpoints.* The reported `dapo` and `grpo` rows correspond to step-150 checkpoints rather than the final training step, since the reward and length curves had already plateaued. A more careful early-stopping protocol could shift the headline numbers slightly.
- *Failure analysis.* The four-way taxonomy in Table 9 is qualitative; a quantitative count over a larger inspected pool would make the priorities for future reward additions sharper.

9.4 Future work

Two concrete extensions follow naturally from Section 8: a precondition-aware reward to attack *overconfident heuristic shortcuts* on `TheoremQA`, and a state-consistency micro-reward to attack *event-boundary* errors on `GSM8K`. Both require an external verifier richer than the current final-answer check, but the current pipeline cleanly accommodates an extra reward component without changing the rest of the stack.

Reproducibility. All code, prompts, hyperparameter scripts, evaluation configurations, and saved samples are kept in the project repository under `training/`, `rewards/`, `verl_bridge/`, and `eval/`. The complete result table is in `EVAL_RESULT_TABLE_20260414.md`, and the training-progress narrative is in `PROJECT_REPORT.md`.

References

[1] Qwen Team. *Qwen2.5 Technical Report*. Open weights at https://huggingface.co/Qwen/Qwen2.5-1.5B. 2024. arXiv: 2412.15115 [cs.CL]. URL: https://arxiv.org/abs/2412.15115.

[2] Z. Shao et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. Introduces GRPO: Group Relative Policy Optimisation. 2024. arXiv: 2402.03300 [cs.CL]. URL: https://arxiv.org/abs/2402.03300.

[3] Q. Yu et al. *DAPO: An Open-Source LLM Reinforcement Learning System at Scale*. 2025. arXiv: 2503.14476 [cs.CL]. URL: https://arxiv.org/abs/2503.14476.

[4] Z. He et al. *DeepMath-103K: A Large-Scale, Challenging, Decontaminated, and Verifiable Mathematical Dataset for Advancing Reasoning*. 2025. arXiv: 2504.11456 [cs.LG]. URL: https://arxiv.org/abs/2504.11456.

[5] K. Cobbe et al. "Training Verifiers to Solve Math Word Problems". In: 2021. arXiv: 2110.14168 [cs.LG]. URL: https://arxiv.org/abs/2110.14168.

[6] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. "Measuring Mathematical Problem Solving with the MATH Dataset". In: *NeurIPS Datasets and Benchmarks Track*. 2021. URL: https://arxiv.org/abs/2103.03874.

[7] H. Lightman et al. *Let's Verify Step by Step*. Defines the 500-problem MATH-500 evaluation subset used in this report. 2023. arXiv: 2305.20050 [cs.LG]. URL: https://arxiv.org/abs/2305.20050.

[8] W. Chen et al. "TheoremQA: A Theorem-driven Question Answering dataset". In: *EMNLP*. 2023. URL: https://arxiv.org/abs/2305.12524.

[9] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. "HellaSwag: Can a Machine Really Finish Your Sentence?" In: *ACL*. 2019. URL: https://arxiv.org/abs/1905.07830.

[10] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. "Measuring Massive Multitask Language Understanding". In: *ICLR*. 2021. URL: https://arxiv.org/abs/2009.03300.

[11] DeepSeek-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. 2025. arXiv: 2501.12948 [cs.CL]. URL: https://arxiv.org/abs/2501.12948.

[12] L. Gao et al. *The Language Model Evaluation Harness*. 2024. DOI: 10.5281/zenodo.12608602. URL: https://github.com/EleutherAI/lm-evaluation-harness.

[13] G. Sheng et al. *verl: Volcano Engine Reinforcement Learning for LLMs*. Open-source RL framework used in this project for GRPO and DAPO runs. 2024. URL: https://github.com/volcengine/verl.